



Session 19:

Spring Security với Access Token, Refresh Token



- 1. So sánh Access Token và Refresh Token**
- 2. Hướng dẫn cấu hình Access Token**
- 3. Hướng dẫn cấu hình Refresh Token**
- 4. Hướng dẫn cấu hình Revoke Token**

1

So sánh Access Token và Refresh Token

Khái niệm: Access Token vs Refresh Token

Access Token

- Token **ngắn hạn** (thường 5–30 phút)
- Dùng để gọi các API được bảo vệ
- Chứa thông tin xác thực và phân quyền (claims)
- Gửi trong header: `Authorization: Bearer <token>`
- Là JWT — phi trạng thái, không lưu server

Refresh Token

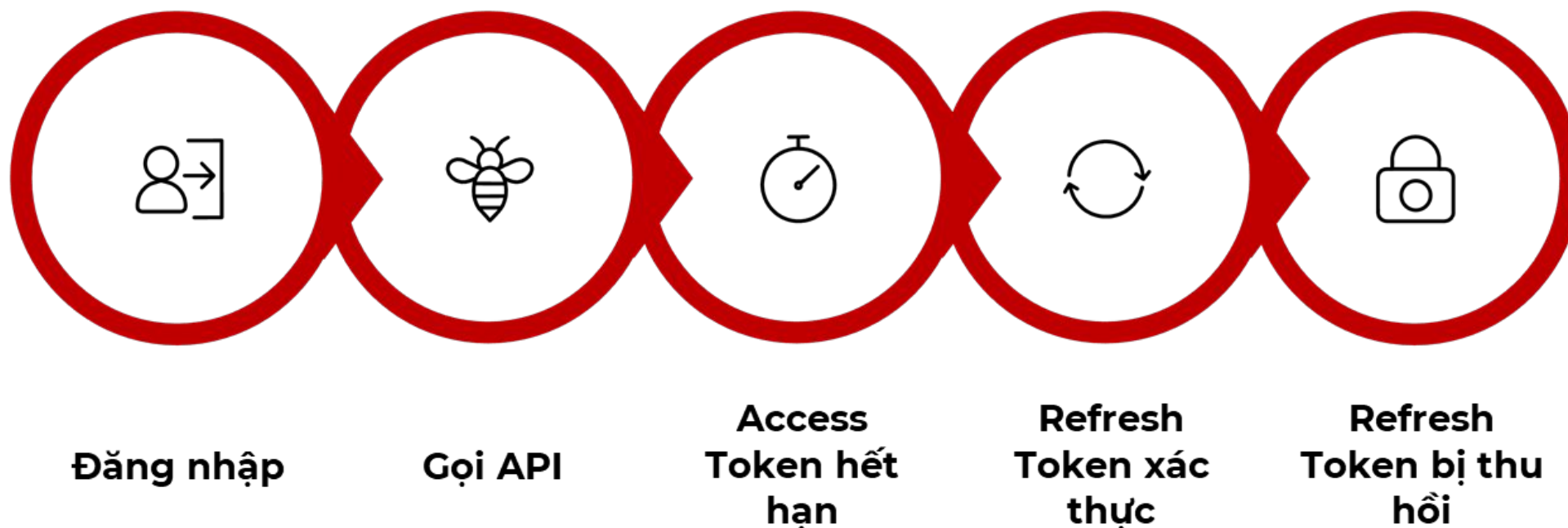
- Token **dài hạn** (vài ngày đến vài tháng)
- Dùng để lấy Access Token mới khi hết hạn
- Chỉ gửi đến endpoint `/refresh-token`
- Lưu phía client: httpOnly cookie hoặc secure storage
- Lưu phía server (DB) để có thể thu hồi

So sánh: Access Token vs Refresh Token

Đặc điểm	Access Token	Refresh Token
Vòng đời	Ngắn (giảm rủi ro nếu bị lộ)	Dài (duy trì phiên làm việc)
Tần suất gửi	Mỗi request API	Chỉ khi Access Token hết hạn
Lưu trữ client	Memory, localStorage	HttpOnly Cookie, Secure Storage
Rủi ro nếu có	Cao nhưng hết hạn nhanh	Rất cao -> cần revoke kịp thời
Có thể thu hồi	Không trực tiếp (phi trạng thái)	Có - Nếu lưu server-side

⚠ Refresh Token có vòng đời dài nên là mục tiêu tấn công hấp dẫn hơn. Luôn lưu và kiểm soát chặt chẽ phía server!

Luồng hoạt động cơ bản



Luồng này là nền tảng của mọi ứng dụng web hiện đại sử dụng xác thực không trạng thái. Access Token xử lý phần lớn các request, trong khi Refresh Token đảm bảo người dùng không cần đăng nhập lại quá thường xuyên.

2

Hướng dẫn cấu hình

Access Token

Các bước cấu hình Access Token

1

Thêm Dependencies

spring-boot-starter-security,
jjwt-api, jjwt-impl, jjwt-jackson,
spring-boot-starter-data-jpa

2

Tạo TokenProvider

Lớp tiện ích để sinh token
(generateAccessToken), parse
(getUserName) và xác thực
(validateJwtToken).

3

Cấu hình application.properties

Thiết lập app.jwt.secret và
app.jwt.accessExpirationMs
cho thời gian hết hạn.

4

Tạo AuthenticationFilter

Filter kế thừa OncePerRequestFilter, đọc
token từ header, xác thực và set
SecurityContext.

5

Cập nhật SecurityFilterChain

Đăng ký filter vào chain bằng addFilterBefore
trước UsernamePasswordAuthenticationFilter.

Code: Sinh Access Token với TokenProvider

```
public String generateAccessToken(String username) {  
    return Jwts.builder()  
        .setSubject(username)  
        .setIssuedAt(new Date())  
        .setExpiration(new Date(  
            System.currentTimeMillis() + accessExpirationMs))  
        .signWith(SignatureAlgorithm.HS512, jwtSecret)  
        .compact();  
}  
public String getUserNameFromJwtToken(String token) {  
    return Jwts.parser()  
        .setSigningKey(jwtSecret)  
        .parseClaimsJws(token)  
        .getBody()  
        .getSubject();  
}  
public boolean validateJwtToken(String token) { ... }  
}
```

Code: AuthenticationFilter

```
public class AuthenticationFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        try {
            String jwt = parseJwt(request);
            if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
                String username = jwtUtils.getUserNameFromJwtToken(jwt);
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(username, null, new ArrayList<>());
                SecurityContextHolder.getContext()
                    .setAuthentication(auth);
            }
        } catch (Exception e) {
            // Log lỗi, không throw để request tiếp tục
        }
        filterChain.doFilter(request, response);
    }
}
```

Cấu hình SecurityFilterChain

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {
        http.csrf().disable()
        .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**")
        .permitAll()
        .anyRequest().authenticated()
        )
        .addFilterBefore(
            jwtAuthenticationFilter(),
            UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

3

Hướng dẫn cấu hình Refresh Token

Cách tiếp cận Refresh Token trong Spring Boot

1. Lưu tại Server (DB)

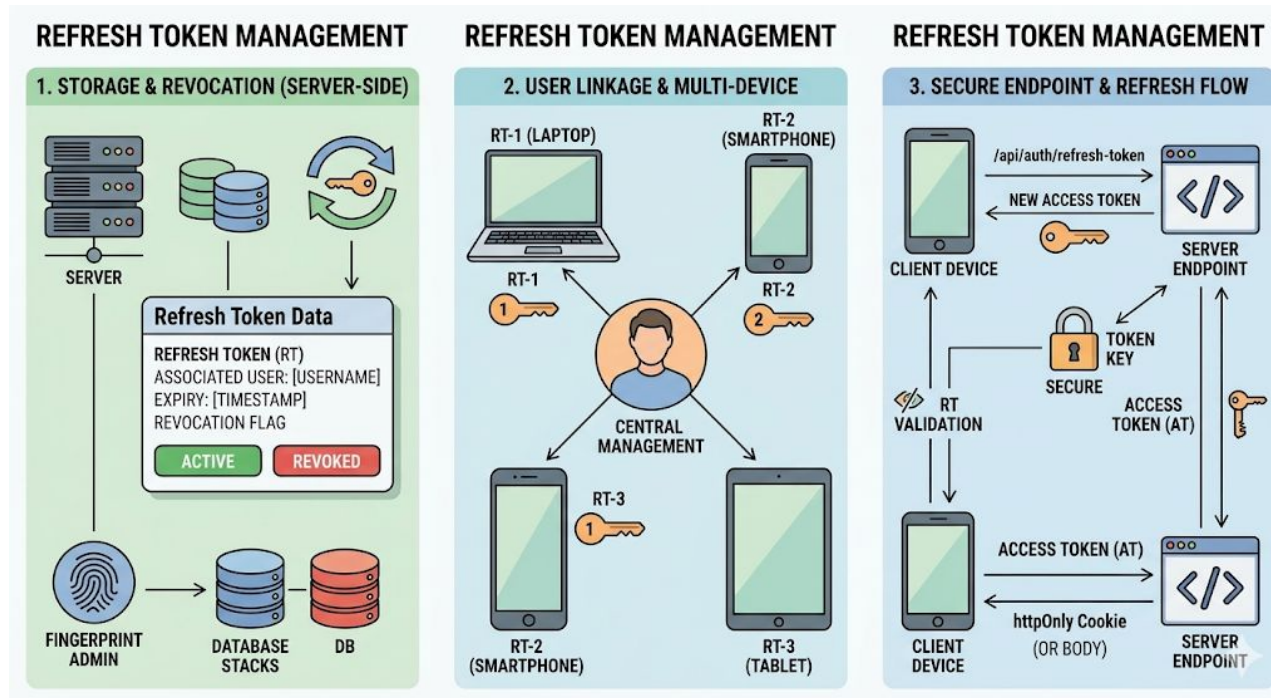
Refresh Token được lưu trong DB, gắn với từng user, có thời gian hết hạn riêng và cờ revoked để thu hồi bất kỳ lúc nào.

2. Liên kết với User

Mỗi Refresh Token gắn với một username cụ thể. Một user có thể có nhiều Refresh Token (nhiều thiết bị), tất cả đều quản lý được.

3. Endpoint riêng biệt

Khi gọi tới Endpoint `/api/auth/refresh-token` sẽ nhận lại Refresh Token (qua body hoặc httpOnly cookie) và trả về Access Token mới.



Entity & Repository cho Refresh Token

```
@Entity
public class RefreshToken {
    @Id @GeneratedValue
    private Long id;
    private String token;
    private String username;
    private Instant expiryDate;
    private boolean revoked;
}
```

```
public interface RefreshTokenRepository
    extends JpaRepository<RefreshToken, Long> {
    Optional<RefreshToken> findByToken(String token);
    // Xóa tất cả token của một user (khi logout all)
    void deleteByUsername(String username);
    // Lấy tất cả token chưa revoke của user
    List<RefreshToken> findByUsername(String username);
}
```

📌 Nên thêm index trên cột `token` và `username` trong database để tăng hiệu năng truy vấn khi tải lớn.

Service: Sinh và xác thực Refresh Token

```
@Service
public class RefreshTokenService {
    @Value("${app.jwt.refreshExpirationMs}")
    private Long refreshExpirationMs;

    // Tạo và lưu Refresh Token mới
    public RefreshToken createRefreshToken(String username) {
        RefreshToken refreshToken = new RefreshToken();
        refreshToken.setUsername(username);
        refreshToken.setToken(UUID.randomUUID().toString());
        refreshToken.setExpiryDate(
            Instant.now().plusMillis(refreshExpirationMs));
        refreshToken.setRevoked(false);
        return refreshTokenRepository.save(refreshToken);
    }
}
```

Service: Sinh và xác thực Refresh Token

```
@Service
public class RefreshTokenService {
    // Kiểm tra hạn sử dụng
    public RefreshToken verifyExpiration(RefreshToken token) {
        if (token.getExpiryDate().compareTo(Instant.now()) < 0) {
            token.setRevoked(true);
            refreshTokenRepository.save(token);
            throw new RuntimeException("Refresh token đã hết hạn");
        }
        if (token.isRevoked()) {
            throw new RuntimeException("Refresh token đã bị thu hồi");
        }
        return token;
    }
}
```


Cấp Access Token mới & cập nhật Login

Endpoint /refresh-token

```
@PostMapping("/refresh-token")
public ResponseEntity<?> refreshToken(@RequestBody RefreshTokenRequest req) {
    String tokenStr = req.getRefreshToken();
    RefreshToken refreshToken = refreshTokenService.findByToken(tokenStr)
        .orElseThrow(() -> new RuntimeException("Token không tồn tại"));

    // Kiểm tra hết hạn và revoked
    refreshTokenService.verifyExpiration(refreshToken);

    // Cấp Access Token mới
    String newAccessToken = jwtUtils.generateAccessToken(refreshToken.getUsername());
    return ResponseEntity.ok(new AccessTokenResponse(newAccessToken));
}
```

Cấp Access Token mới & cập nhật Login

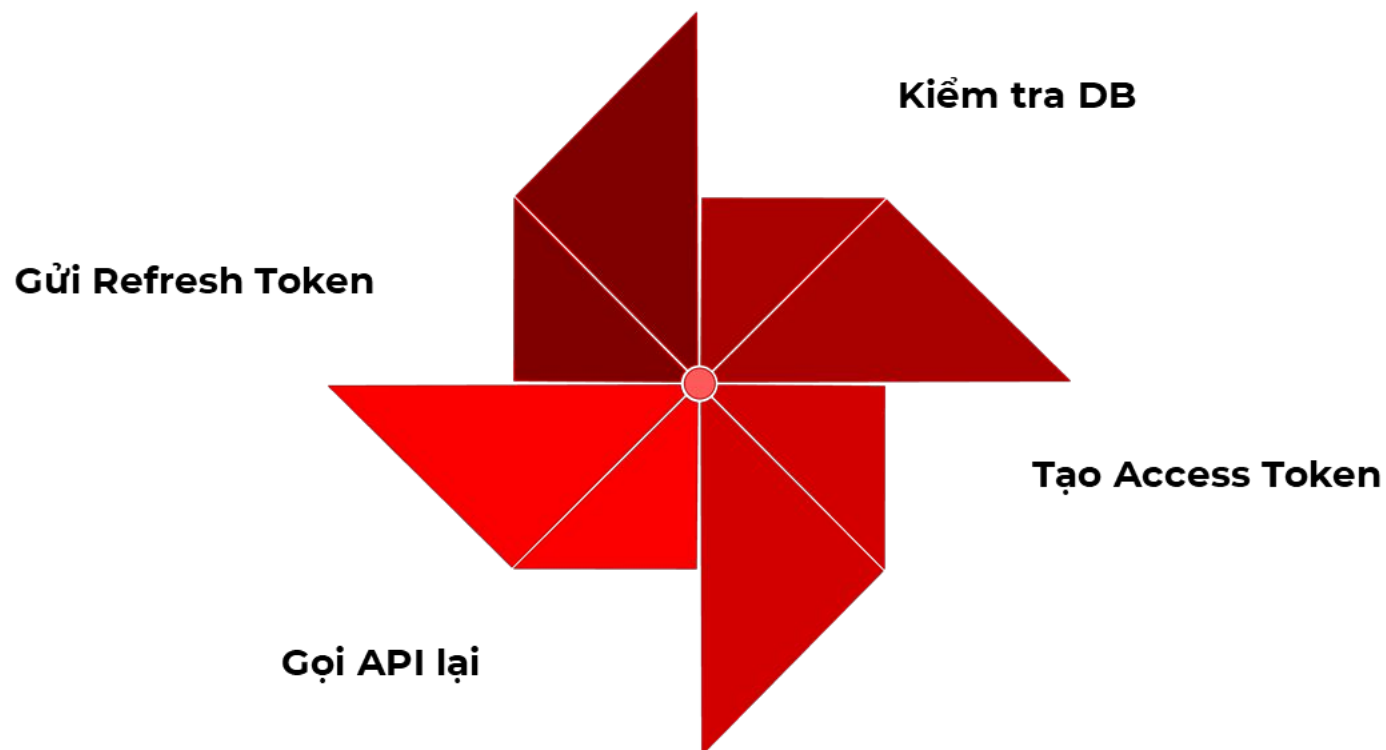
Cập nhật Login Response

Khi login thành công, server trả về **cả hai token**:

- **Access Token** → trả trong JSON response body
- **Refresh Token** → nên gửi qua **httpOnly cookie** để bảo mật hơn
- Lưu **Refresh Token** vào DB bằng **createRefreshToken()**

✔ httpOnly cookie không thể đọc bằng JavaScript — ngăn chặn tấn công XSS đánh cắp Refresh Token một cách hiệu quả.

Luồng Refresh Token hoàn chỉnh



Luồng này diễn ra trong nền, hoàn toàn minh bạch với người dùng. Client cần tự động phát hiện lỗi 401 (Unauthorized) và gọi endpoint refresh trước khi retry request ban đầu — thường xử lý trong HTTP interceptor (Axios, Fetch).

4

Hướng dẫn cấu hình

Revoke Token

Khi nào cần Revoke Token

1

Đăng xuất

Người dùng chủ động đăng xuất — cần vô hiệu hóa Refresh Token ngay lập tức.

2

Phát hiện token bị lộ

Hệ thống hoặc người dùng báo cáo token bị đánh cắp — revoke ngay toàn bộ.

3

Đổi mật khẩu

Khi mật khẩu thay đổi, tất cả token cũ phải bị vô hiệu hóa để tránh bị lạm dụng.

4

Quản trị viên khóa tài khoản

Admin vô hiệu hóa tài khoản người dùng, cần revoke tất cả token đang hoạt động.

Cơ chế Revoke cho Access Token

Access Token là JWT phi trạng thái — **không thể revoke trực tiếp**. Có ba giải pháp thực tế:

1

Blacklist Token

Lưu jti (JWT ID) vào bảng đen cho đến khi token hết hạn. Có thể dùng **Redis** với TTL tương ứng để tự động xóa.

2

Thời gian hết hạn ngắn

Kết hợp Access Token (5–15 phút) với Refresh Token. Khi cần revoke, chỉ cần revoke Refresh Token — Access Token sẽ tự hết hạn sớm.

3

Token Version (nâng cao)

Mỗi user có tokenVersion trong DB. Nhúng version vào JWT claim. Khi revoke, tăng version → tất cả token cũ không hợp lệ.

Revoke Refresh Token & Token Version

Revoke Refresh Token (cơ bản)

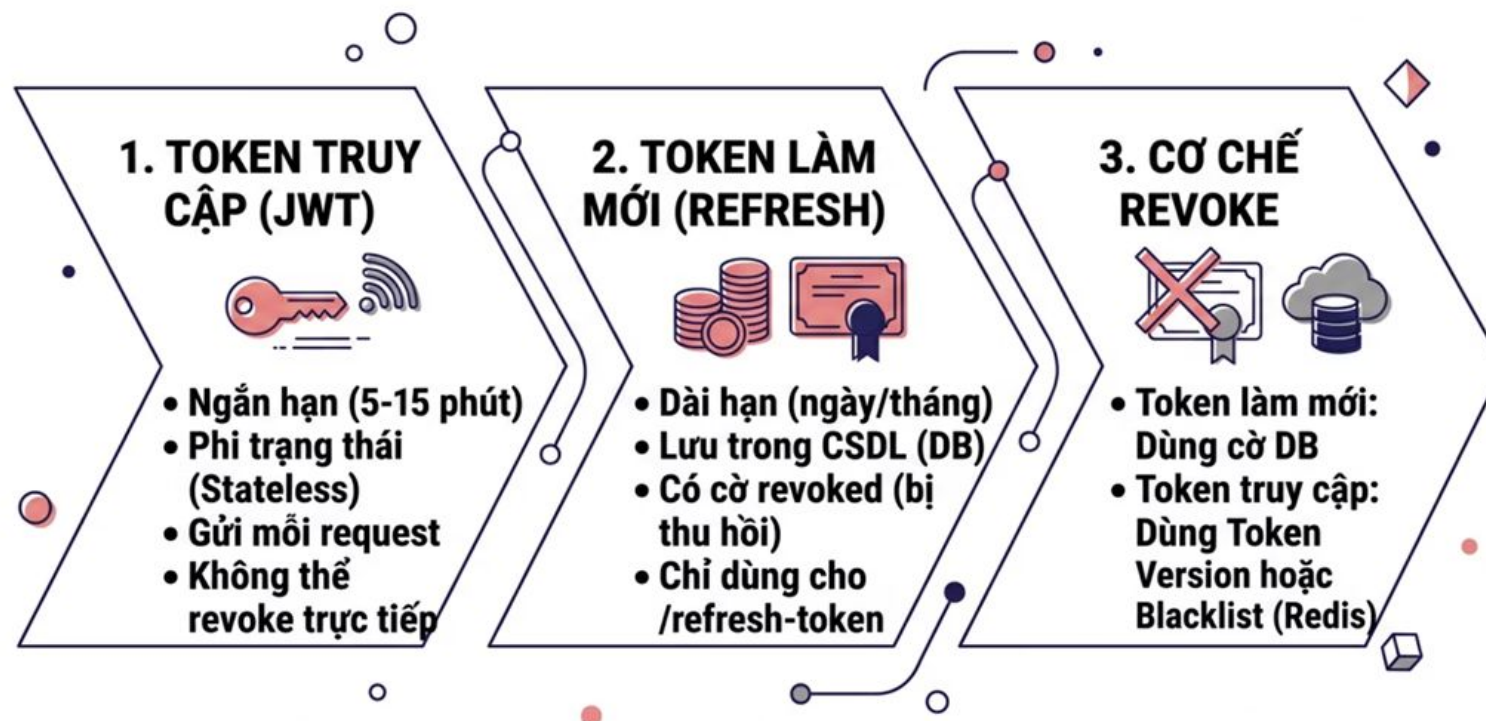
```
// Đánh dấu tất cả token của user là revoked
public void revokeAllUserTokens(String username) {
    List<RefreshToken> userTokens =
        refreshTokenRepository.findByUsername(username);
    userTokens.forEach(t -> t.setRevoked(true));
    refreshTokenRepository.saveAll(userTokens);
}

// Gọi trong logout và change password
@PostMapping("/logout")
public ResponseEntity<?> logout(
    @CookieValue("refreshToken") String refreshToken) {
    refreshTokenService.revokeToken(refreshToken);
    // Xóa httpOnly cookie trên client
    return ResponseEntity.ok("Đã đăng xuất");
}
```

Token Version (nâng cao)

```
// Trong JwtAuthenticationFilter
String username = jwtUtils
    .getUserNameFromJwtToken(jwt);
// Lấy version từ custom claim "ver"
int tokenVersion = jwtUtils
    .getVersionFromJwtToken(jwt);
User user = userService
    .findByUsername(username);
// So sánh version
if (tokenVersion != user.getTokenVersion()) {
    throw new JwtException(
        "Token đã bị thu hồi");
}
// Khi revoke: tăng tokenVersion lên 1
user.setTokenVersion(
    user.getTokenVersion() + 1);
userRepository.save(user);
```

Kiến trúc Token hoàn chỉnh



✓ **Khuyến nghị thực tế:** Luôn kết hợp Access Token ngắn hạn + Refresh Token có thể revoke để cân bằng giữa hiệu năng (stateless) và bảo mật (có thể kiểm soát).

- ❑ Access Token là JWT ngắn hạn, phi trạng thái.
- ❑ Access Token gửi mọi request qua Authorization: Bearer ...
- ❑ Access Token không revoke trực tiếp được
- ❑ Refresh Token là JWT dài hạn, lưu trên DB server
- ❑ Refresh Token chỉ dùng để lấy Access Token mới.
- ❑ Refresh Token có thể revoke bằng cờ revoked = true
- ❑ Nên áp dụng đủ 3 cơ chế Access Token, Refresh Token, Revoke Token trong các ứng dụng thực tế



KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN